## Swapping of values

Python allows swapping of the values of two variables in a single statement without having to use a third variable, like in other languages:

```
>>> num1, num2 = 10, 20
>>> print (num1, num2)
10 20
>>>

>>> num1, num2 = num2, num1
>>> print (num1, num2)
20 10
>>>
```

## Combining a list into a string

A list of values can be concatenated into a single string by adding the elements of the list using a loop. But this is inefficient, particularly if the list is long. A Python string is immutable. Hence, a new string is created during every iteration. The same can be done easily by using the *join()* function:

```
>>> namelist = [“ram”, “shyam”,
“rahul”, “ravi”, “ashok”]
>>> namestr = “”.join(namelist)
>>> print (namestr)
ramshyamrahulraviashok
>>>
```

## Printing a list of strings with comma as a separator

Printing a list of strings as output separated by a comma can be done easily in the following way:

```
>>> names = [“Anand”, “Rahul”,
“Sairam”, “Shiva”]
>>> print (*names, sep = ‘,’)
Anand,Rahul,Sairam,Shiva
>>>
```

The ‘*’ in the above line is used for unpacking an iterable (a list in this case). The *sep* parameter specifies the character to use to separate the objects in the list. The default is ‘ ‘ (space).

## Creating a list of unique elements

Removing duplicates in a list is easy using the set data structure, as the latter does not allow duplicate elements. The list can be converted to sets, which automatically eliminates duplicates. The set is converted back to the list and the new list will then have only unique elements.

```
>>> numlist = [1,1,2,2,3,3,3,4,4,5,5]
>>> numlist = list (set (numlist))
>>> print (numlist)
[1, 2, 3, 4, 5]
>>>
```

## The interactive ‘_’ operator

The ‘_’ (single underscore) is used as a temporary variable to save the result of the last executed expression only in the Python interactive shell, and its value can be printed or used in other ways.

```
>>> a = 8
>>> b = 7
>>> a * b
56
>>>_
56
>>>
```

## Returning multiple values from a function

Functions defined in Python can return multiple values unlike languages like C/C++ or Java, which allow only a single value to be returned. The multiple values are returned as a tuple.

```
>>> #sample of Python function
returning multiple values
>>> def test():
      a = 1
      b = 2
      c = 3
        return a, b, c

>>> d, e, f  = test()
>>> print (d, e, f)
1 2 3
>>>
```

## Chaining of comparison operators

Python allows the use of multiple comparisons in a single expression by chaining the comparison operators. The expression returns *True* only if all the comparisons are true and returns *False* otherwise.

```
>>> a = 10
>>> 5 < a < 20
True
>>>
```

The above is equal to writing ( a > 5) and (a < 20).

Another example is given below:

```
>>> 20 < a < 50
False
```

The above is equal to writing ( a > 20) and (a < 50).

## Use of ternary operator for conditional assignment

Like many other programming languages, Python also has a facility that gives the effect of a ternary operator. Python defines a conditional expression, which helps to convert the *if..else* statement to a one-line conditional expression. This results in making the code concise due to multiple lines being reduced to a single line, which is also easier to read. The syntax is:

```
<expression1> if <condition> else
<expression>
```

The condition is evaluated and if the result is *True*, expression1 is evaluated; if the result of the condition is *False*, expression2 is evaluated.

```
>>> num = 55
>>> “odd” if num % 2 == 1 else “even”
‘odd’
>>>
```

A condition having three possibilities can be written using nested *if else* in a